

COLLISCOPE

Workload-Aware Evaluation and Scope-Aware Cuckoo Hashing for Compiler Symbol Tables

MASTER AGENT EXECUTION MANUAL

Prepared for: Google Antigravity (coding / development agent)

Repository: <https://github.com/swapniljain2024-ai/ColliScope>

Academic topic: Performance Evaluation of Compiler Symbol Table Implementations — Collision Resolution

This document is an engineering execution manual, not a project report. It contains a reusable project-context block, global operating rules, a phased development plan (Phase 0–11), and self-contained, copy-paste-ready prompts for each phase. Paste the PROJECT CONTEXT block at the start of every new Antigravity session, then paste the prompt for the phase currently in progress.

How to Use This Document

1. Start every new Antigravity session by pasting the "PROJECT CONTEXT" block below. It never changes between phases.
2. Then paste the ANTIGRAVITY PROMPT for the phase you are currently executing. Do not paste multiple phase prompts into a single session unless a phase explicitly depends on running back-to-back.
3. Each phase prompt is self-contained: it tells Antigravity what to inspect, what to build, what not to touch, what must pass, what to commit, and what message to use.
4. Use the BUG FIX / REFACTOR / ARCHITECTURE REVIEW / DOCUMENTATION templates at the end for anything that falls outside the phase plan.
5. Do not let Antigravity skip ahead. Each phase ends with a verification gate and a push gate. The next phase should only start after those gates are satisfied.

Project Context

PROJECT: ColliScope
FORMAL TITLE: "ColliScope: Workload-Aware Evaluation and Scope-Aware Cuckoo Hashing for Compiler Symbol Tables"
ACADEMIC TOPIC: Performance evaluation of a compiler's Symbol Table implementation - Collision Resolution
REPOSITORY: <https://github.com/swapniljain2024-ai/ColliScope>

CORE IDEA

This project has TWO connected contributions:

1. A workload-aware experimental framework for evaluating collision-resolution techniques using compiler-like workloads.
2. SVC-Hash - our proposed scope-aware extension of cuckoo hashing, designed specifically for compiler symbol-table operations.

RESEARCH STORY: CHARACTERIZE -> ISOLATE -> EXPLAIN -> DESIGN -> VALIDATE

PIPELINE:

REAL SOURCE
-> WORKLOAD EXTRACTION
-> WORKLOAD MATRIX
-> IDENTICAL OPERATION TRACES
-> MULTIPLE COLLISION-RESOLUTION IMPLEMENTATIONS
-> SVC-HASH
-> BENCHMARKING
-> STATISTICAL ANALYSIS
-> INTERACTIVE RESEARCH DASHBOARD
-> RESEARCH REPORT / PAPER

WE ARE NOT BUILDING:

- A generic "symbol table implementation" class project
- A compiler
- A claim that SVC-Hash is universally faster
- A claim that we invented cuckoo hashing
- A claim about what "modern compilers" universally use
- A claim of novelty merely because no exact search result was found

THE THREE CONTRIBUTIONS:

1. Workload contribution - systematically characterize compiler-like workloads by identifier distribution/frequency and lexical scope structure.
2. Technical contribution - design and implement SVC-Hash, a scope-aware cuckoo-based symbol-table structure.
3. Evaluation contribution - use identical compiler-style traces to compare all implementations under controlled conditions, including a critical Plain Cuckoo vs SVC-Hash ablation.

ALGORITHMS (benchmark set):

BASELINES:

1. Separate Chaining
2. Cuckoo Hashing
3. Hopscotch Hashing

(Coalesced Hashing is an OPTIONAL 5th baseline, only if it introduces no meaningful implementation risk or delay.)

PROPOSED:

4. SVC-Hash

EXPLICITLY EXCLUDED (faculty has already ruled these out - never reintroduce them):

- Linear Probing
- Quadratic Probing
- Double Hashing
- Robin Hood Hashing

The three baselines are INDEPENDENT comparison implementations, not development stages. SVC-Hash is the actual proposed system. The single most important comparison in the whole project is:

PLAIN CUCKOO HASHING vs SVC-HASH

because it isolates exactly what scope-awareness adds to a cuckoo-based design.

SVC-HASH CONCEPT

Treat SVC-Hash as a serious data-structure implementation, not a mock or demo. Conceptually: a scope-aware cuckoo-based symbol table. Investigate and use as appropriate:

- flat bucketized storage
- two hash functions / cuckoo candidate locations
- symbol identifier + associated symbol info/value
- scope_id, active/inactive scope state, lexical scope relationships
- correct shadowing
- scope entry / scope exit
- per-scope ownership/lifecycle tracking
- tombstones for removed symbols if required
- stash/overflow handling if justified
- rehash/rebuild/compaction where required

A prior design sketch considered: bucket size 4, two hash functions, a small overflow stash, scope identifiers, a scope interval / Euler-style ancestor representation, per-scope insertion logs, tombstoning at scope exit, and selecting the visible declaration per lexical scope rules. These are a STARTING POINT, not unquestionable requirements. Antigravity must inspect the repository and the accumulated design decisions first, and pick the safest technically correct implementation. Any deviation from the prior sketch must be documented with a reason.

CORRECTNESS COMES BEFORE PERFORMANCE, always. SVC-Hash must correctly handle: nested scopes, shadowing, scope exit, lookup of outer declarations, undeclared identifiers, and repeated declarations per the chosen semantic assumptions (assumptions must be written down explicitly wherever they are made).

WORKLOAD MATRIX (must not depend only on random identifiers)

Identifier dimension: Random | Real-source | Frequency-matched synthetic

Scope dimension: Flat | Nested

Workload type: Declaration-heavy | Lookup-heavy | Mixed

Load factor examples: 0.25, 0.40, 0.55, 0.70, 0.80, 0.90
(Make these configurable rather than hard-coded wherever practical.)

Research question: "How does collision-resolution performance change when the workload starts resembling the actual behavior of compiler symbol tables?"

Also investigate: identifier reuse, non-uniform identifier frequencies, scope nesting, shadowing, declaration/reference ratio, lookup frequency, load factor, and the interaction between frequency and scope.

TRACE FORMAT (illustrative - the exact format must be specified precisely and be machine-readable; do not change it silently once other phases depend on it):

```
ENTER_SCOPE 1
DECLARE count
ENTER_SCOPE 2
DECLARE count
ENTER_SCOPE 3
DECLARE value
REFERENCE count
EXIT_SCOPE 3
EXIT_SCOPE 2
REFERENCE count
EXIT_SCOPE 1
```

CRITICAL FAIRNESS REQUIREMENT

Every algorithm (Chaining, Cuckoo, Hopscotch, SVC-Hash) must receive the EXACT SAME trace for a given experiment. Never generate different workloads per algorithm. This is essential for experimental validity.

TECH STACK

- Performance-critical implementations + benchmark engine: C++
- Workload extraction, trace generation, experiment orchestration, statistical analysis, visualization: Python
- Dashboard: Streamlit + Plotly (or another sensible Python visualization stack) - must be a POLISHED research tool, not a default-looking Streamlit app.
- Keep the architecture modular. Give each algorithm a common interface where practical (insert/declaration, lookup/reference, enterScope, exitScope, reset/clear, metrics retrieval) - but never force a bad abstraction just to make interfaces identical.

METRICS

Common: insertion time, lookup time, throughput, memory usage, collision count, load factor.

Chaining-specific: chain length, comparisons, traversal length.

Cuckoo-specific: relocations/kicks, rehashes, maximum displacement.

Hopscotch-specific: neighbourhood movements, comparisons.

SVC-Hash-specific: candidate locations examined, scope checks, scope-exit cost, stash/overflow usage (if implemented), relocations, rebuild/rehash behaviour, correctness statistics.

Do NOT pretend "probe count" means the same thing across algorithms. Include latency (avg and p99) where practical.

CORRECTNESS TESTING (mandatory, minimum coverage): global declaration +

lookup; nested scopes; shadowing; exiting inner scope; lookup after scope exit; multiple nested shadowing levels; undeclared identifier; repeated references; collision-heavy inputs; rehash/rebuild situations; randomized scope traces; comparison of SVC-Hash lookup results against a trusted reference/oracle implementation. Correctness must be verified BEFORE any performance claim is trusted.

STATISTICAL EVALUATION

Repeated trials (~30 where practical for important experiments). Report mean, standard deviation, confidence intervals where practical, median where useful, p95/p99 where useful, effect sizes where appropriate. Use warm-up runs. Keep hardware/compiler configuration fixed and recorded. Avoid misleading graphs.

RESEARCH INTEGRITY (non-negotiable)

Always distinguish IMPLEMENTED FACT from HYPOTHESIS from EXPERIMENTAL RESULT from INTERPRETATION. Never hard-code a conclusion before running the experiment that would support it (do not write "SVC-Hash is faster than all algorithms" - instead write things like "We evaluate whether SVC-Hash provides stable lookup behaviour under increasing lexical nesting depth"). Never claim "no one has ever done this." If a literature review is performed later, report exactly what was found and cite it. Never fabricate benchmark results, literature citations, or experimental conclusions. If the dashboard has no real run for a given configuration, it must say so - never display fabricated numbers.

REAL-DATA PROVENANCE - NON-NEGOTIABLE

Any workload labelled REAL must originate directly from an actual, documented open-source C/C++ repository. Do not fabricate, randomly generate, paraphrase, or reconstruct identifiers, frequencies, scopes, declarations, references, or traces and label them as REAL. For every REAL corpus, preserve: repository URL, exact commit hash/version, license, source-file list, extraction configuration, extraction statistics, and the resulting raw extracted dataset/trace. The repository must contain or reference sufficient artifacts to demonstrate the provenance of the data used in reported experiments. Synthetic workloads are permitted only when explicitly labelled SYNTHETIC or FREQUENCY-MATCHED SYNTHETIC and must never be represented as real-source observations.

Keep this distinction impossible to miss:

- REAL: actual source code -> actual extracted data -> actual trace
- SYNTHETIC: actual source statistics -> controlled synthetic workload (legitimate, but must be labelled as such, never as REAL)

GIT / GITHUB WORKFLOW (mandatory)

Repository: <https://github.com/swapniljain2024-ai/ColliScope>

- Work incrementally, phase by phase - never wait until the whole project is "done" to push.
- After each phase: implement -> test -> check for regressions -> clean up -> update docs -> meaningful commit -> push -> verify the pushed state.
- Never use meaningless commit messages ("fixed typo", "update", "final", "final2", "test123"). Use descriptive messages tied to what was actually built in that phase.

- Before pushing, confirm the code builds and tests pass. Never push knowingly broken code unless a phase explicitly documents an experimental branch/state as such.

WHEN ANTIGRAVITY HITS A PROBLEM

Stop. Diagnose. Explain the issue in plain terms. Propose the safest solution. Implement only after that proposal is validated (either by the user, or - for small, low-risk, clearly-scoped issues - after stating the reasoning inline and proceeding cautiously). Never hide errors, silently change scope, or silently change the trace/interface format that other phases depend on.

Global Antigravity Operating Rules

These rules apply to every phase prompt in this document. Antigravity should treat them as standing instructions layered on top of each individual phase prompt.

For every phase, Antigravity must know:

- What to inspect first (repository state, prior phase outputs, existing docs).
- What files/components to create or modify.
- What NOT to modify unnecessarily.
- What functionality must exist after the phase.
- What tests must pass.
- What evidence/output should be produced (files, logs, screenshots, CSVs).
- How to verify the phase is actually complete.
- How to update documentation.
- How to commit (and with what message).
- How to push, and how to verify the push succeeded.
- What the next phase depends on from this one.

Hard rules

- Never generate the entire project in one shot - work phase by phase, incrementally.
- Never remove SVC-Hash, the workload matrix, the ablation study, correctness testing, phase-by-phase Git commits, or the dashboard - these are core requirements, not optional.
- Never replace the dashboard with terminal-output-only demos.
- Never simplify this into a generic "symbol table implementation" student project.
- Never hard-code fake performance numbers - all dashboard numbers must come from real experiment output files or real execution.
- Never reintroduce Linear Probing, Quadratic Probing, Double Hashing, or Robin Hood Hashing - they have been explicitly ruled out.

Phased Development Plan

Antigravity may split any phase below into smaller sub-phases if it proves too large for a single incremental unit of work - but must not skip phases, reorder core dependencies, or collapse multiple phases into a single uncommitted burst of work.

Phase	Focus
PHASE 0	Repository & Architecture Audit
PHASE 1	Core Infrastructure
PHASE 2	SVC-Hash Core
PHASE 3	Baseline Implementations
PHASE 4	Trace System
PHASE 5	Workload Generation
PHASE 6	Benchmark Engine
PHASE 7	Validation + Ablation
PHASE 8	Statistical Analysis
PHASE 9	ColliScope Research Dashboard
PHASE 10	Final Integration
PHASE 11	Research Report Support

PHASE 0: Repository & Architecture Audit

Depends on: None. This is the entry point for the whole project.

You are Antigravity, the development agent for the ColliScope project (see PROJECT CONTEXT above - treat it as binding for this entire session and every session after it).

PHASE 0 GOAL: Understand exactly what exists in the repository today, and establish (or confirm) the architecture, build/test system, README, and Git workflow - without assuming anything about the current state.

STEP 1 - INSPECT BEFORE TOUCHING ANYTHING

1. Clone/open <https://github.com/swapniljain2024-ai/ColliScope>.
2. Enumerate: branches, commit history, README contents, existing source files, existing configuration (build files, CI, package manifests, .gitignore), and any existing docs/ or results/.
3. Do NOT assume the repository is empty. Do NOT assume any particular existing structure. Report what you actually find before proposing anything.
4. Summarize findings in a short AUDIT.md (or append to docs/AUDIT.md) describing: current state, what is reusable, what conflicts with the PROJECT CONTEXT requirements, and what is missing.

STEP 2 - PROPOSE / CONFIRM ARCHITECTURE

Propose a repository layout adapted from (not blindly copied from) this reference shape, reconciled with whatever already exists:

```
ColliScope/
|-- src/
| |-- chaining/
| |-- cuckoo/
| |-- hopscotch/
| |-- svc_hash/
| `-- common/
|-- workloads/
| |-- traces/
| |-- real_source/
| `-- generators/
|-- benchmarks/
|-- tests/
|-- analysis/
|-- dashboard/
|-- results/
|-- docs/
`-- README.md
```

Explain any deviation from this reference shape in AUDIT.md.

STEP 3 - ESTABLISH BUILD / TEST SYSTEM

- Set up the C++ build system (CMake recommended unless the repo already standardizes on something else) with a place for each

algorithm module and the benchmark engine.

- Set up the Python environment (requirements.txt or pyproject.toml) for workload/trace/analysis/dashboard code.
- Add a minimal "hello world" test on each side (one C++ test target, one Python test) purely to prove the toolchain works end-to-end.

STEP 4 - README AND GIT WORKFLOW

- Write/update README.md with: project name, one-paragraph description (pulled from PROJECT CONTEXT, not invented), repo structure, and build/test instructions.
- Confirm git remote, default branch, and commit workflow. Do not create new branches unless the existing repo state requires it for safety (e.g. isolating a risky experiment).

WHAT NOT TO DO

- Do not implement any algorithm logic yet.
- Do not invent project requirements beyond PROJECT CONTEXT.
- Do not delete existing repository content without first documenting why it conflicts with the required architecture.

TESTS THAT MUST PASS

- C++ build succeeds from a clean checkout.
- The placeholder C++ test target runs and passes.
- The placeholder Python test runs and passes (pytest).

EVIDENCE TO PRODUCE

- docs/AUDIT.md (repository audit findings).
- Updated README.md.
- Working, minimal build/test scaffolding on both sides.

VERIFY BEFORE COMMITTING

- Fresh clone builds and both placeholder tests pass.
- README accurately reflects the repo as it now stands.

COMMIT & PUSH

- Commit message: "Initialize ColliScope architecture"
- Push to the existing default branch and confirm the push landed (check the remote state, not just local success).

NEXT PHASE DEPENDENCY

Phase 1 (Core Infrastructure) depends on the directory layout, build system, and test scaffolding established here being stable.

PHASE 1: Core Infrastructure

Depends on: Phase 0's repository layout, build system, and test scaffolding.

PHASE 1 GOAL: Build the shared infrastructure every algorithm and every later phase will depend on: common interfaces, configuration, metrics infrastructure, the trace format contract, and the test framework.

STEP 1 - INSPECT FIRST

Re-read docs/AUDIT.md and the current src/common/ (if any). Confirm Phase 0's scaffolding still builds before adding anything.

STEP 2 - COMMON INTERFACE (src/common/)

Define a common C++ interface that each algorithm module (chaining, cuckoo, hopscotch, svc_hash) will implement where practical:

- insert/declaration(key, value, scope_id)
- lookup/reference(key, scope_id) -> value or "not found"
- enterScope(scope_id, parent_scope_id)
- exitScope(scope_id)
- reset/clear()
- metrics retrieval (returns a structured metrics object, see below)

Document explicitly where this interface will NOT fit an algorithm cleanly (e.g. Chaining/Cuckoo/Hopscotch may not need scope-awareness at all in their base form) and how that will be handled - do not force a bad abstraction. This decision must be written down in docs/architecture.md.

STEP 3 - CONFIGURATION SYSTEM

Define one configuration mechanism (e.g. JSON or YAML config files) used consistently across: workload generation, trace generation, the benchmark engine, and the dashboard. Configuration must cover, at minimum: identifier dimension, scope dimension, workload type, load factor, operation mix, algorithm selection, and number of trial repetitions.

STEP 4 - METRICS INFRASTRUCTURE

Define a metrics schema (C++ struct + matching JSON/CSV serialization) covering the common metrics from PROJECT CONTEXT (insertion time, lookup time, throughput, memory usage, collision count, load factor) plus extension points for algorithm-specific metrics. Document the schema in docs/metrics.md, including an explicit note that "probe count" and similar terms are NOT directly comparable across algorithms.

STEP 5 - TRACE FORMAT CONTRACT

Formally specify the trace format (see PROJECT CONTEXT for the illustrative example) in docs/trace_format.md: exact grammar, allowed operations (ENTER_SCOPE, EXIT_SCOPE, DECLARE, REFERENCE, and any others needed), and how ambiguous cases (e.g. reference to an undeclared identifier) are represented. This format is a contract that Phases 3, 4, 5, and 6 all depend on - treat changes to it after this phase as breaking changes requiring explicit justification.

STEP 6 - TEST FRAMEWORK

Set up the C++ test framework (e.g. GoogleTest or Catch2 - pick one and document why) and the Python test framework (pytest). Confirm both are wired into the build/CI path from Phase 0.

WHAT NOT TO DO

- Do not implement any actual collision-resolution logic yet.
- Do not lock in SVC-Hash-specific fields in the common interface if they don't generalize - keep the common interface genuinely common.

TESTS THAT MUST PASS

- Unit tests for config loading (valid + invalid config cases).
- Unit tests for metrics struct serialization/deserialization.
- Unit tests (or a parser stub + tests) for trace format parsing, including at least one malformed-trace rejection case.

EVIDENCE TO PRODUCE

- docs/architecture.md, docs/metrics.md, docs/trace_format.md.
- Working config loader, metrics schema, and trace-format parser stub with passing tests.

VERIFY BEFORE COMMITTING

- All new unit tests pass; Phase 0 tests still pass (no regressions).

COMMIT & PUSH

- Commit message: "Add core infrastructure: interfaces, config, metrics, trace contract"

NEXT PHASE DEPENDENCY

Phase 2 (SVC-Hash core) and Phase 3 (baselines) both implement the common interface defined here and emit metrics using the schema defined here.

PHASE 2: SVC-Hash Core

Depends on: Phase 1's common interface, metrics schema, and config system.

PHASE 2 GOAL: Implement SVC-Hash itself - the actual proposed data structure - as a serious, correctness-first implementation.

STEP 1 - INSPECT FIRST

Re-read the SVC-HASH CONCEPT section of PROJECT CONTEXT and docs/architecture.md from Phase 1. Confirm the common interface still fits; if it needs a scope-aware extension, propose the extension and document why in docs/architecture.md before writing implementation code.

STEP 2 - DESIGN DECISION LOG (WRITE BEFORE CODING)

Create docs/svc_hash_design.md. For each of the following, state the decision and the reasoning, explicitly marking anything that deviates from the prior design sketch in PROJECT CONTEXT:

- bucket size and layout (flat bucketized storage)
- number/choice of hash functions and candidate-location strategy
- how scope_id, active/inactive scope state, and lexical scope relationships are represented
- how shadowing is resolved at lookup time
- how scope entry/exit are implemented (including cost model)
- tombstone strategy for removed symbols, if used
- stash/overflow handling, if used, and why it is or isn't needed
- rehash/rebuild/compaction triggers and behavior

STEP 3 - IMPLEMENT SVC-HASH (src/svc_hash/)

Implement insertion/declaration, lookup/reference, enterScope, and exitScope against the design log above. Correctness comes before performance - do not micro-optimize before the correctness suite (Step 4) passes.

STEP 4 - CORRECTNESS TESTS (mandatory, minimum set)

Implement tests for: global declaration + lookup; nested scopes; shadowing; exiting an inner scope; lookup after scope exit; multiple nested shadowing levels; undeclared identifier; repeated references; collision-heavy inputs (force many collisions and confirm correctness, not just that it doesn't crash); rehash/rebuild situations; randomized scope traces (generate random valid trace sequences and check invariants hold). Where practical, cross-check SVC-Hash's lookup result against a simple reference/oracle symbol-table implementation (a plain scoped map is sufficient) on the same randomized traces.

STEP 5 - METRICS HOOKS

Wire SVC-Hash-specific metrics (candidate locations examined, scope checks, scope-exit cost, stash/overflow usage if implemented, relocations, rebuild/rehash behaviour, correctness statistics) into the Phase 1 metrics schema.

WHAT NOT TO DO

- Do not claim or write anywhere that "SVC-Hash is faster" - no benchmarking has happened yet.
- Do not skip the oracle cross-check tests to save time.
- Do not silently change the common interface from Phase 1 without updating docs/architecture.md.

TESTS THAT MUST PASS

- The full correctness suite listed in Step 4, with zero known-failing cases (mark any skipped edge case explicitly with a reason, don't silently omit it).

EVIDENCE TO PRODUCE

- docs/svc_hash_design.md (design decisions + reasoning).
- src/svc_hash/ implementation.
- tests/svc_hash/ correctness suite, all passing, with output log captured under results/phase2_correctness_log.txt.

VERIFY BEFORE COMMITTING

- Full correctness suite green. Phases 0-1 tests still pass.

COMMIT & PUSH

- Commit message: "Implement SVC-Hash core storage and scope management"

NEXT PHASE DEPENDENCY

Phase 7 (validation + ablation) directly depends on SVC-Hash being correct before any performance claims are evaluated. Phase 3 (baselines) can proceed in parallel/independently.

PHASE 3: Baseline Implementations

Depends on: Phase 1's common interface and metrics schema.

PHASE 3 GOAL: Implement the three baseline collision-resolution techniques as independent comparison implementations (not development stages toward SVC-Hash).

STEP 1 - INSPECT FIRST

Re-read docs/architecture.md and confirm the common interface from Phase 1. Confirm Phase 2 did not silently change that interface in an incompatible way; if it did, reconcile before proceeding.

STEP 2 - IMPLEMENT SEPARATE CHAINING (src/chaining/)

Standard chaining implementation against the common interface. Metrics: chain length, comparisons, traversal length, plus the common metrics set.

STEP 3 - IMPLEMENT CUCKOO HASHING (src/cuckoo/)

Plain cuckoo hashing (two hash functions, standard eviction/kick behavior, rehash on cycle/failure). This implementation is the direct comparison point for SVC-Hash in Phase 7's ablation - keep it a clean, non-scope-aware baseline. Metrics: relocations/kicks, rehashes, maximum displacement, plus common metrics.

STEP 4 - IMPLEMENT HOPSCOTCH HASHING (src/hopscotch/)

Standard hopscotch hashing with a defined neighbourhood size. Metrics: neighbourhood movements, comparisons, plus common metrics.

STEP 5 - (OPTIONAL) COALESCED HASHING

Only implement if it introduces no meaningful implementation risk or schedule delay. If skipped, note the decision in docs/architecture.md. Do NOT implement Linear Probing, Quadratic Probing, Double Hashing, or Robin Hood Hashing under any circumstances - they are explicitly excluded.

STEP 6 - CORRECTNESS TESTS

For each baseline: basic insert/lookup correctness, correctness under forced collisions, and (for cuckoo) correctness across rehash events. Baselines do not need scope-awareness tests unless the common interface requires them to expose enterScope/exitScope as no-ops - if so, test that they behave as documented no-ops.

WHAT NOT TO DO

- Do not add scope-awareness logic to the baselines - they must stay clean, unmodified baseline implementations so the SVC-Hash comparison is fair and interpretable.
- Do not skip the plain-cuckoo baseline's correctness tests - Phase 7's ablation depends on this implementation being trustworthy.

TESTS THAT MUST PASS

- Correctness suite for each of the three (or four) baselines, all

passing.

EVIDENCE TO PRODUCE

- src/chaining/, src/cuckoo/, src/hopscotch/ (and optionally src/coalesced/), each with its own test suite and passing results captured under results/phase3_correctness_log.txt.

VERIFY BEFORE COMMITTING

- All baseline correctness suites green; Phases 0-2 tests still pass.

COMMIT & PUSH

- Commit message: "Implement baseline collision-resolution algorithms (Chaining, Cuckoo, Hopscotch)"

NEXT PHASE DEPENDENCY

Phase 6 (benchmark engine) and Phase 7 (ablation) both require all baseline implementations plus SVC-Hash to share the common interface and metrics schema.

PHASE 4: Trace System

Depends on: Phase 1's trace format contract (docs/trace_format.md).

PHASE 4 GOAL: Build the trace format's parser, executor, a reference (oracle) implementation, and deterministic replay - the mechanism that guarantees every algorithm receives the exact same operation sequence.

STEP 1 - INSPECT FIRST

Re-read docs/trace_format.md (Phase 1). If Phase 2 or 3 surfaced a need to extend the format, reconcile that here explicitly and update docs/trace_format.md with a changelog entry - do not change the format silently.

STEP 2 - TRACE PARSER

Implement a parser (Python for authoring/inspection tooling, C++ for the benchmark-engine-facing path) that reads a trace file into a structured, ordered operation list, and rejects malformed traces with a clear error.

STEP 3 - TRACE EXECUTOR

Implement an executor that replays a parsed trace, deterministically, against any implementation of the common interface (from Phase 1). This is the mechanism that enforces the CRITICAL FAIRNESS REQUIREMENT: the exact same parsed trace object must be replayed against Chaining, Cuckoo, Hopscotch, and SVC-Hash without any per-algorithm variation.

STEP 4 - REFERENCE / ORACLE IMPLEMENTATION

Implement (or promote SVC-Hash's Phase 2 oracle) a simple, obviously-correct scoped symbol table (e.g. a stack of hash maps) to serve as the semantic ground truth for REFERENCE operation results. The executor should be able to run any trace against the oracle and capture expected lookup results.

STEP 5 - DETERMINISTIC REPLAY VALIDATION

Implement a check that runs the same trace object through the executor against two different algorithms and asserts: (a) the sequence of operations issued to each was byte-identical, and (b) every REFERENCE operation's result matches the oracle's result for both algorithms.

WHAT NOT TO DO

- Do not generate or mutate traces inside the executor - trace generation belongs to Phase 5. The executor only replays.
- Do not let per-algorithm code paths see a different trace ordering.

TESTS THAT MUST PASS

- Parser tests (valid + malformed traces).
- Executor determinism test (identical operation sequence across algorithms).
- Oracle-agreement test on at least one hand-written trace with known expected lookup results, and one randomized trace set from Phase 2's randomized-trace generator.

EVIDENCE TO PRODUCE

- Trace parser + executor + oracle implementation, with tests, and a short docs/trace_system.md explaining how to use them.
- A sample trace file checked into workloads/traces/ with its expected oracle output.

VERIFY BEFORE COMMITTING

- All trace-system tests pass; Phases 0-3 tests still pass.

COMMIT & PUSH

- Commit message: "Add trace-driven execution engine with oracle validation"

NEXT PHASE DEPENDENCY

Phase 5 (workload generation) produces traces in this exact format.

Phase 6 (benchmark engine) uses this executor to run every experiment.

PHASE 5: Workload Generation

Depends on: Phase 4's trace format and parser.

PHASE 5 GOAL: Build the Python-based workload extraction/generation pipeline that produces traces across the full workload matrix (identifier dimension x scope dimension x workload type x load factor).

STEP 1 - INSPECT FIRST

Re-read the WORKLOAD MATRIX and REAL SOURCE -> TRACE PIPELINE sections of PROJECT CONTEXT, and docs/trace_format.md from Phase 4.

STEP 2 - RANDOM WORKLOAD GENERATOR

Generate traces with random identifiers, configurable scope depth (flat vs nested), configurable declaration/reference ratio, and configurable target load factor.

STEP 3 - REAL-SOURCE EXTRACTION

Build a pipeline that extracts identifier names, identifier frequencies, declaration/reference events, lexical scope structure, scope nesting depth, and shadowing relationships (where reliably detectable) from real C/C++ source files (only files that are legally appropriate to use - document the source and license of any corpus used in docs/workload_sources.md). Use a proper parser/library approach (e.g. an existing C/C++ parsing library or tool) rather than fragile regex-based extraction, if a reasonable one is available; if none is available within scope/time, document that decision explicitly and describe the extraction approach's known limitations.

PROVENANCE (non-negotiable, see PROJECT CONTEXT): for every REAL corpus used, record in docs/workload_sources.md the repository URL, exact commit hash/version, license, source-file list, extraction configuration, and extraction statistics, alongside the raw extracted dataset. Never let a REAL-labelled trace be generated, paraphrased, or reconstructed rather than actually extracted.

STEP 4 - FREQUENCY-MATCHED SYNTHETIC GENERATOR

Generate synthetic traces whose identifier frequency distribution is fit to match the distribution observed in the real-source extraction from Step 3 (e.g. matching a Zipf-like curve), without reusing the real identifiers verbatim.

STEP 5 - WORKLOAD MATRIX ASSEMBLY

Build a generator/orchestrator that, given a config (per Phase 1's config system), produces the full cross-product of: identifier dimension x scope dimension x workload type x load factor - all serialized as trace files in workloads/traces/, with a manifest (JSON/CSV) describing each trace's parameters.

STEP 6 - VALIDATION

Validate every generated trace against the Phase 4 parser (must parse cleanly) and confirm the achieved load factor / scope depth / ratio matches the requested configuration within a documented tolerance.

WHAT NOT TO DO

- Do not generate different traces for different algorithms under the same experiment configuration - one trace per configuration, reused for every algorithm (enforced later by Phase 6/7).
- Do not silently drop the real-source or frequency-matched dimension if it proves difficult - document blockers and the fallback taken.

TESTS THAT MUST PASS

- Generator unit tests for each dimension (random, real-source, frequency-matched) producing parseable, schema-valid traces.
- A test that the workload matrix assembler produces the full expected cross-product for a small example config.

EVIDENCE TO PRODUCE

- workloads/generators/ implementation.
- A populated workloads/traces/ directory (or a documented subset, if full generation is expensive) with a manifest.
- docs/workload_sources.md documenting any real-source corpus used.

VERIFY BEFORE COMMITTING

- All generator tests pass; every generated trace parses cleanly under Phase 4's parser; Phases 0-4 tests still pass.

COMMIT & PUSH

- Commit message: "Add real-source workload extraction and workload matrix generation"

NEXT PHASE DEPENDENCY

Phase 6 (benchmark engine) consumes the traces and manifest produced here as its experiment input.

PHASE 6: Benchmark Engine

Depends on: Phase 3's algorithm implementations, Phase 4's executor, Phase 5's traces.

PHASE 6 GOAL: Build the controlled experiment runner that executes every algorithm against every configured trace, with repetitions, and emits structured metrics output.

STEP 1 - INSPECT FIRST

Re-read docs/metrics.md (Phase 1), the trace executor (Phase 4), and the workload manifest produced in Phase 5.

STEP 2 - EXPERIMENT RUNNER

Implement a benchmark engine (C++ core, optionally Python-orchestrated) that, given an experiment config, runs a specified set of algorithms against a specified set of traces, for a configurable number of repetitions, with warm-up runs, and enforces the CRITICAL FAIRNESS REQUIREMENT (same parsed trace object/replay for every algorithm within a given run).

STEP 3 - METRICS OUTPUT

Emit results as CSV and/or JSON (per Phase 1's metrics schema), including: experiment configuration, algorithm identity, trace identity, per-trial raw metrics, and environment metadata (hardware, compiler flags, date/commit hash of the code that produced the run).

STEP 4 - CONFIGURABILITY

Make the benchmark matrix configurable rather than hard-coded: which algorithms, which traces/workload-matrix cells, load factors, operation mixes, and repetition count must all be config-driven (per Phase 1's config system). Do not create thousands of unnecessary experiment configurations solely to produce more graphs - keep the default matrix aligned with PROJECT CONTEXT's stated dimensions.

STEP 5 - SANITY BENCHMARK RUN

Run a small representative subset end-to-end (all algorithms, a handful of trace configurations, small repetition count) and confirm output files are produced correctly and look sane (no NaNs, no obviously broken timing).

WHAT NOT TO DO

- Do not hard-code the benchmark matrix in a way that requires code changes to adjust load factor or workload selection.
- Do not report fabricated or estimated numbers if a run fails - surface the failure.

TESTS THAT MUST PASS

- Unit tests for the runner's config parsing and repetition/warm-up logic.
- An integration test running one small end-to-end benchmark and validating the output file schema.

EVIDENCE TO PRODUCE

- benchmarks/ implementation.
- results/phase6_sanity_run/ containing the sanity benchmark's raw output (CSV/JSON) and environment metadata.

VERIFY BEFORE COMMITTING

- Sanity run completes without errors and produces schema-valid output; Phases 0-5 tests still pass.

COMMIT & PUSH

- Commit message: "Add trace-driven benchmark engine with configurable experiment matrix"

NEXT PHASE DEPENDENCY

Phase 7 (validation + ablation) and Phase 8 (statistical analysis) both consume this benchmark engine's output format.

PHASE 7: Validation + Ablation

Depends on: Phase 2 (SVC-Hash), Phase 3 (baselines, esp. plain Cuckoo), Phase 6 (benchmark engine).

PHASE 7 GOAL: Run the full correctness validation pass and the mandatory Plain Cuckoo vs SVC-Hash ablation study, plus the scope-depth and load-factor experiment sweeps.

STEP 1 - INSPECT FIRST

Re-read the ABLATION STUDY and CORRECTNESS TESTING sections of PROJECT CONTEXT, and confirm Phase 2's and Phase 3's correctness suites are still green before running any performance comparison.

STEP 2 - CORRECTNESS VALIDATION PASS

Re-run the full correctness suites (Phase 2 SVC-Hash, Phase 3 baselines) plus any new randomized/cross-algorithm correctness checks enabled by the trace system (Phase 4): every algorithm's lookup results on the same trace must agree with the oracle. Record results in results/phase7_correctness_validation.txt. Do not proceed to Step 3 until this is fully green.

STEP 3 - PLAIN CUCKOO VS SVC-HASH ABLATION (mandatory)

Using the benchmark engine (Phase 6), run Plain Cuckoo Hashing and SVC-Hash against the SAME scope-heavy traces (nested scope, shadowing-heavy configurations from the workload matrix). Measure and report:

- overhead scope-awareness introduces (time, memory)
- benefit scope-awareness provides (if any, and under what conditions)
- whether lookup behaviour changes with nesting depth
- memory overhead
- scope-exit cost
- correctness under shadowing (should already be validated in Step 2, but explicitly re-confirm in this context)

Report trade-offs honestly - do not conclude SVC-Hash is simply "faster"; report what the data actually shows, including cases where SVC-Hash is slower or has overhead.

STEP 4 - SCOPE-DEPTH EXPERIMENTS

Sweep scope depth (e.g. 1, 4, 8, 16, 32, 64 - adjust based on actual implementation feasibility, document any limit reached) and measure how each algorithm's relevant metrics change.

STEP 5 - LOAD-FACTOR AND OPERATION-MIX EXPERIMENTS

Run the full workload-matrix sweep (identifier dimension x scope dimension x workload type) at each configured load factor, for all algorithms, using the benchmark engine's configured matrix from Phase 6.

WHAT NOT TO DO

- Do not skip or shortcut the ablation study - it is the single most important experiment in the project.
- Do not report a conclusion ("SVC-Hash is better/worse") without the

underlying data file backing it up in results/.

TESTS THAT MUST PASS

- Correctness validation pass fully green (Step 2) before any ablation numbers are treated as trustworthy.

EVIDENCE TO PRODUCE

- results/phase7_correctness_validation.txt
- results/ablation_cuckoo_vs_svc/ (raw CSV/JSON output)
- results/scope_depth_sweep/ and results/workload_matrix_sweep/
- docs/ablation_findings.md summarizing findings using IMPLEMENTED FACT / HYPOTHESIS / EXPERIMENTAL RESULT / INTERPRETATION labeling.

VERIFY BEFORE COMMITTING

- All correctness suites green; ablation and sweep runs completed with real output files (not placeholders).

COMMIT & PUSH

- Commit message: "Run correctness validation and Plain Cuckoo vs SVC-Hash ablation study"

NEXT PHASE DEPENDENCY

Phase 8 (statistical analysis) consumes the raw results produced here.
Phase 9 (dashboard) will surface these exact result files - not new or re-derived numbers.

PHASE 8: Statistical Analysis

Depends on: Phase 7's raw experiment output.

PHASE 8 GOAL: Turn the raw benchmark output into a rigorous statistical analysis pipeline: aggregation, confidence intervals, p99, effect sizes, and publication-quality plots.

STEP 1 - INSPECT FIRST

Re-read the STATISTICAL EVALUATION section of PROJECT CONTEXT and confirm the raw result files from Phase 7 are present and complete (expected number of trials per configuration, ~30 where practical).

STEP 2 - AGGREGATION PIPELINE (analysis/)

Build a Python pipeline that loads the raw CSV/JSON output and computes, per (algorithm, workload configuration): mean, standard deviation, confidence intervals (where practical), median (where useful), p95/p99 (where useful), and effect sizes (where appropriate, e.g. comparing SVC-Hash vs Plain Cuckoo).

STEP 3 - PUBLICATION-QUALITY PLOTS

Generate plots (matplotlib/plotly, saved as static images for the report and reusable data for the dashboard) covering: algorithm comparison across the workload matrix, scope-depth sweep results, and the Plain Cuckoo vs SVC-Hash ablation with error bars/confidence intervals shown honestly (not misleading axes/truncation).

STEP 4 - REPRODUCIBILITY

Ensure the analysis pipeline is re-runnable from the raw result files alone (no manual steps), and record exactly which commit/run produced which analysis output.

WHAT NOT TO DO

- Do not compute statistics on anything other than the real Phase 7 output files.
- Do not produce plots with misleading scales, cherry-picked ranges, or omitted error bars where variance is material.

TESTS THAT MUST PASS

- Unit tests for the aggregation functions (mean/stdev/CI/percentile calculations) against known small synthetic inputs with hand-computed expected values.

EVIDENCE TO PRODUCE

- analysis/ pipeline code.
- results/analysis/ containing aggregated tables (CSV) and plots (PNG/HTML) for: algorithm comparison, scope-depth sweep, and the ablation.

VERIFY BEFORE COMMITTING

- Aggregation unit tests pass; pipeline runs end-to-end from Phase 7's raw output and produces all listed plots/tables without manual intervention.

COMMIT & PUSH

- Commit message: "Add statistical analysis pipeline and publication-quality plots"

NEXT PHASE DEPENDENCY

Phase 9 (dashboard) reads directly from results/analysis/ (and/or the raw Phase 7 output) - it must not recompute or duplicate this logic.

PHASE 9: ColliScope Research Dashboard

Depends on: Phase 7's raw results and Phase 8's analysis output.

PHASE 9 GOAL: Build the polished, research-tool-quality interactive dashboard. This is a major part of the final demo - it must not look like a default Streamlit app.

STEP 1 - INSPECT FIRST

Re-read the WEB DASHBOARD and UI QUALITY REQUIREMENT sections of PROJECT CONTEXT in full before writing any UI code. Re-confirm the exact file paths of results/analysis/ and the raw Phase 7 output, since the dashboard must only ever display real data from these files.

STEP 2 - INFORMATION ARCHITECTURE

Implement these sections as distinct, clearly-navigable views:

1. Overview / Home
2. Workload Explorer
3. Algorithm Comparison
4. SVC-Hash / Scope Visualizer
5. Benchmark Runner
6. Results / Analytics
7. Correctness / Trace Viewer

STEP 3 - VISUAL IDENTITY

Establish a consistent ColliScope visual identity: clean navigation, clear hierarchy, KPI/metric cards, interactive Plotly charts, readable tables, and a clear, consistent visual distinction between BASELINES and PROPOSED (SVC-Hash) throughout every view. Avoid unnecessary animation and visual clutter. Design for a responsive layout where practical.

STEP 4 - INTERACTIONS

Implement: choosing a workload; choosing algorithms to compare; choosing load factor and scope depth; choosing operation mix; running or selecting a benchmark; comparing algorithms; inspecting metrics; viewing a raw trace; visualizing the scope hierarchy; demonstrating shadowing; viewing the SVC vs Cuckoo ablation; inspecting individual benchmark run details.

STEP 5 - KILLER DEMO: SCOPE/SHADOWING VISUALIZER

Implement the interactive shadowing demonstration described in PROJECT CONTEXT (nested DECLARE x at multiple scope levels, LOOKUP resolving to the innermost active declaration, EXIT_SCOPE changing what LOOKUP resolves to). Visually show the active scope stack and which declaration a given lookup currently resolves to.

STEP 6 - NO FAKE DATA, HONEST EMPTY STATES

Every number shown must originate from a real Phase 7/Phase 8 output file. If a given workload/algorithm/config combination has not actually been benchmarked, the UI must say so clearly (a defined empty state), never silently substitute a placeholder or estimated number. Implement

clear loading and error states throughout.

WHAT NOT TO DO

- Do not ship a default, unstyled Streamlit look.
- Do not hard-code any performance numbers anywhere in the dashboard code.
- Do not blur the BASELINES vs PROPOSED distinction.

TESTS THAT MUST PASS

- Smoke tests that each of the 7 views renders without error against the real Phase 7/8 output.
- A test confirming the empty-state path triggers correctly for a workload/algorithm combination with no corresponding result file.

EVIDENCE TO PRODUCE

- dashboard/ implementation.
- Screenshots of all 7 views saved under docs/screenshots/ (for later use in the research report).

VERIFY BEFORE COMMITTING

- Dashboard launches cleanly, all 7 views render against real data, empty/loading/error states verified manually.

COMMIT & PUSH

- Commit message: "Build ColliScope research dashboard"

NEXT PHASE DEPENDENCY

Phase 10 (final integration) wires this dashboard into the end-to-end reproducible pipeline and validates it against a clean install.

PHASE 10: Final Integration

Depends on: All prior phases (0-9).

PHASE 10 GOAL: Produce a clean, reproducible, end-to-end pipeline from a fresh checkout, with complete documentation and a rehearsed demo workflow.

STEP 1 - INSPECT FIRST

Re-read docs/AUDIT.md, docs/architecture.md, and every phase's evidence artifacts to confirm nothing is missing or silently broken.

STEP 2 - CLEAN INSTALLATION TEST

From a genuinely fresh clone (or an equivalent clean environment), run through: build -> run correctness tests -> run (or reuse) a benchmark subset -> run the analysis pipeline -> launch the dashboard. Document the exact commands in docs/quickstart.md. Fix anything that breaks in a clean environment.

STEP 3 - END-TO-END PIPELINE SCRIPT

Provide a single orchestration script (or a short documented sequence of commands) that takes the project from "fresh clone" to "dashboard running with real results," suitable for a faculty demo machine.

STEP 4 - DOCUMENTATION PASS

Ensure README.md and docs/ collectively cover: project overview, architecture, how to build, how to run tests, how to generate workloads/traces, how to run benchmarks, how to run analysis, how to launch the dashboard, and where to find prior phase design decisions (svc_hash_design.md, ablation_findings.md, etc.).

STEP 5 - DEMO REHEARSAL

Walk through the FINAL DEMO FLOW (see the Final Demo Checklist section of this document) end-to-end on the clean environment from Step 2 and record it (screenshots or a short script of exact clicks/commands) in docs/demo_walkthrough.md.

WHAT NOT TO DO

- Do not leave any step of the pipeline requiring undocumented manual intervention.
- Do not consider this phase complete if any earlier phase's tests are failing.

TESTS THAT MUST PASS

- The full existing test suite (Phases 0-9) passes in the clean environment.
- The end-to-end pipeline script completes without manual intervention.

EVIDENCE TO PRODUCE

- docs/quickstart.md, docs/demo_walkthrough.md.
- A verified clean-install log captured under results/phase10_clean_install_log.txt.

VERIFY BEFORE COMMITTING

- Fresh-environment run-through completed successfully end-to-end.

COMMIT & PUSH

- Commit message: "Finalize end-to-end integration, reproducibility, and documentation"

NEXT PHASE DEPENDENCY

Phase 11 (research report support) pulls final tables/plots and methodology text from this now-stable, reproducible state.

PHASE 11: Research Report Support

Depends on: Phase 8's analysis output and Phase 10's stable, reproducible pipeline.

PHASE 11 GOAL: Generate the final tables, plots, and methodology documentation needed to support the research report/paper - without drawing any new conclusions that aren't backed by Phase 7/8 data.

STEP 1 - INSPECT FIRST

Re-read docs/ablation_findings.md (Phase 7) and results/analysis/ (Phase 8) in full before drafting any report-support material.

STEP 2 - FINAL TABLES AND PLOTS

Produce a final, curated set of report-ready tables and plots (a subset of Phase 8's output, cleaned up for presentation) covering: the workload matrix results, the scope-depth sweep, and the Plain Cuckoo vs SVC-Hash ablation. Save under results/report_assets/.

STEP 3 - METHODOLOGY DOCUMENTATION

Write docs/methodology.md covering: experiment design, workload matrix definition, trace generation approach (including real-source corpus and its license/provenance), benchmark procedure (repetitions, warm-up, environment), and statistical methods used.

STEP 4 - LIMITATIONS

Write docs/limitations.md honestly documenting: known limitations of the workload matrix, known limitations of the real-source extraction, any algorithm-specific caveats, and any experiments that were reduced in scope for feasibility (with the reason).

STEP 5 - REPRODUCIBILITY INFORMATION

Write docs/reproducibility.md with exact commit hash(es), environment (hardware/compiler/OS), and the exact commands to reproduce every figure/table in results/report_assets/.

WHAT NOT TO DO

- Do not introduce new conclusions in this phase that aren't directly traceable to Phase 7/8 result files.
- Do not claim novelty beyond what PROJECT CONTEXT permits (no "no one has ever done this" claims; if literature review exists, cite exactly what was found).

TESTS THAT MUST PASS

- N/A (documentation phase) - but re-run the Phase 10 clean-install check once more to confirm the project is still fully reproducible at hand-off time.

EVIDENCE TO PRODUCE

- results/report_assets/ (final tables/plots).
- docs/methodology.md, docs/limitations.md, docs/reproducibility.md.

VERIFY BEFORE COMMITTING

- Every figure/table in results/report_assets/ traces back to a real file in results/ from Phase 7 or Phase 8.

COMMIT & PUSH

- Commit message: "Add research report assets, methodology, and reproducibility documentation"

NEXT PHASE DEPENDENCY

None - this is the final phase. The repository should now be faculty-demo-ready and report-ready.

Reusable Prompt Templates

Use these for anything that falls outside the phase plan: fixing a bug discovered mid-phase, refactoring before a review, reviewing a component boundary, or writing an Architecture Decision Record.

Bug Fix Template

```
We have a bug: <describe symptom>.
Relevant code: <paste the function/module>.
Walk through the logic step by step for this exact scenario and identify
where behavior diverges from expected. Propose the minimal fix, then
explain the fix. Do not fix anything beyond the described symptom without
flagging it separately first.
```

Refactor Template

```
Here is our current implementation of <component>: <paste code>. It has
grown organically and needs cleanup ahead of Phase <N> / Review <N>.
Preserve all existing test-covered behavior. Propose a refactor plan
FIRST (list of changes + rationale) before rewriting anything.
```

Architecture Review Template

```
Review this component boundary: <paste description/diagram>. Identify any
responsibilities ambiguously shared between the algorithm modules
(chaining/cuckoo/hopscotch/svc_hash), the trace/benchmark engine, and the
dashboard. Suggest a cleaner separation and explain the trade-offs.
```

Documentation Template (ADR)

```
Write an Architecture Decision Record (ADR) for: <decision, e.g. "bucket
size 4 vs 8 for SVC-Hash" or "tombstoning vs compaction at scope exit">.
Include: context, decision, alternatives considered, consequences
(positive and negative).
```

Final Integration Checklist

- Fresh clone builds and all test suites (Phases 0-9) pass.
- Correctness validation (Phase 7, Step 2) is green before any performance number is trusted.
- The Plain Cuckoo vs SVC-Hash ablation exists as real result files, not placeholder numbers.
- The dashboard shows only real data from results/, with honest empty states elsewhere.
- Every commit in the Git history is meaningful and phase-scoped - no "final", "final2", or "fix typo" commits.
- README.md and docs/quickstart.md let a new person reproduce the whole pipeline unaided.
- docs/limitations.md and docs/methodology.md exist and are honest about scope and caveats.

Final Demo Checklist

- Open the ColliScope dashboard.
- Show the project overview.
- Show the workload matrix.
- Select a real/frequency-matched workload.
- Select nested scopes.
- Run or view benchmark results.
- Compare Chaining, Cuckoo, Hopscotch, and SVC-Hash.
- Show scope-depth performance.
- Open the SVC-Hash scope visualizer.
- Demonstrate shadowing.
- Show the Plain Cuckoo vs SVC-Hash ablation.
- Show the underlying trace.
- Show real benchmark result files / terminal output if needed.
- Explain that the research report contains the complete statistical evaluation.

End of document. Paste the PROJECT CONTEXT block at the start of every new Antigravity session, followed by the prompt for the phase currently in progress.